

Decision Trees and Random Forests

From a Single Tree to an Ensemble

Tu Li

Fall 2026

Econ 453 University of Arizona

Roadmap

1. Motivation
2. Anatomy of a Decision Tree
3. Growing a Tree
4. Strengths and Weaknesses of a Single Tree
5. From Trees to a Forest — Bagging
6. Random Forests
7. Summary

1. Motivation

1.1 A Different Kind of Model

Everything we have seen so far fits a **single global formula**:

- OLS / LASSO / ridge (L6, L9, L10): one linear formula $\hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p$, applied to every observation.
- Polynomial regression (L11): one polynomial formula, applied to every observation.
- Logistic regression (L12): one logistic formula, applied to every observation.

But what if the data has **different rules in different regions** of predictor space?

1.2 A Concrete Example

Predict whether a loan will default ($Y = 1$) from two features: `income` (x_1) and `credit_score` (x_2).

The real rule.

- Among low-income borrowers, default rates vary strongly with credit score.
- Among high-income borrowers, default rates stay low even when credit score is lower.

1.2 A Concrete Example 2

A logistic regression of Y on $\beta_0 + \beta_1 \cdot \text{income} + \beta_2 \cdot \text{credit}$ can't easily capture this — it forces one global rule.

We could add interactions ($\beta_3 \cdot \text{income} \times \text{credit}$), but we would need to know in advance which interactions matter. In real applications with many predictors, guessing the right interactions is hopeless.

Decision trees discover the regions and their rules automatically from data.

1.3 The Tree Idea (in Plain English)

A decision tree is a sequence of **yes/no questions** on the predictors, leading to a prediction.

Loan-default example:

1. Is `income` $\leq$ \$40k?

- Yes $\Rightarrow$ ask the next question on credit score.
- No $\Rightarrow$ predict “no default”.

2. Is `credit_score` $\leq$ 600?

- Yes $\Rightarrow$ predict “default”.
- No $\Rightarrow$ predict “no default”.

Three leaves, three predictions. The structure was learned from the data.

1.4 Universal — Regression and Classification

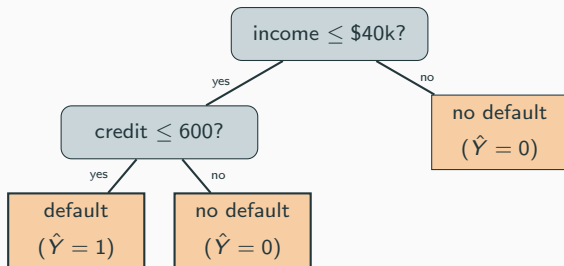
Same machinery for both response types.

- **Classification** (Y categorical). Each leaf predicts a *class label*. Loan default: $\{0, 1\}$. Spam filter: $\{\text{spam}, \text{not spam}\}$.
- **Regression** (Y numeric). Each leaf predicts a *number* — the average of the training y_i in that leaf. House price: predict \$320,000 if the house is in the “downtown, ≤ 3 -bedroom” leaf.

Same algorithm, different prediction rule at the leaf. This is why decision trees are so widely used.

2. Anatomy of a Decision Tree

2.1 What a Tree Looks Like

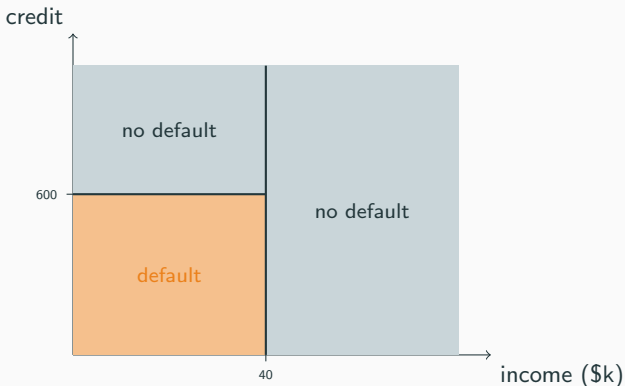


Vocabulary:

- **Root:** the top question.
- **Internal nodes:** questions further down the tree.
- **Leaves:** terminal nodes; each holds a prediction.
- **Split:** the choice of (predictor, threshold) at a node.

2.2 The Tree as a Partition of Space

Each leaf corresponds to a **rectangular region** of predictor space. The tree above carves the (income, credit-score) plane into three boxes:



2.2 The Tree as a Partition of Space 2

Three leaves $\Leftrightarrow$ three regions. In each region, the model predicts a single value. Trees fit a **piecewise constant** function over rectangular partitions.

2.3 Prediction at a Leaf

Once the tree is built, prediction for a new $\mathbf{x}_{\text{new}}$ is straightforward:

1. Drop $\mathbf{x}_{\text{new}}$ down the tree: at each node, check which side of the split it lands on, follow that branch.
2. Read off the prediction at the leaf you reach.

At the leaf, the prediction is:

- **Regression**: the *average* of the training y_i that ended up in this leaf,
$$\hat{y}_{\text{leaf}} = \frac{1}{|\text{leaf}|} \sum_{i \in \text{leaf}} y_i.$$
- **Classification**: the *majority class* among the training y_i in this leaf.

3. Growing a Tree

3.1 The Question — Which Split Is Best?

Building a tree means choosing, at each node, the **split** (predictor X_j , threshold s) that best separates the data.

“Best” means: the two resulting child regions are **more homogeneous** than the parent.

- **Regression**: child regions should have small within-region variance in y .
- **Classification**: child regions should be close to pure (one class dominates).

Same idea, different measure of homogeneity.

3.2 The Split Criterion

Regression split criterion

For a candidate split (X_j, s) at a node, let L and R be the left and right child regions and $\bar{y}_L, \bar{y}_R$ the average response within each. The within-region RSS is

$$\text{RSS}_{\text{split}}(j, s) = \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2.$$

Pick the (j, s) that minimises this quantity.

Classification split criterion

Count how “mixed” each child region is (e.g. how many points are misclassified by the majority-class prediction). Pick the split that makes the two children as **pure** as possible — each child dominated by a single class.

3.2 The Split Criterion 2

Both criteria say the same thing in plain English: *a good split makes the two children look as different as possible from each other, and as similar as possible within themselves.*

“Computer does it.” Software loops over every predictor and every candidate threshold, picks the best.

3.3 Recursive Partitioning

Algorithm: greedy tree growing

Starting from the root (all data):

1. Find the best split (X_j, s) at the current node.
2. Split the data into left/right child regions.
3. Recurse on each child as if it were a fresh root.
4. Stop when a stopping rule is met (next slide).

The algorithm is **greedy**: at each node, it chooses the split that looks best *right now*, without looking ahead. This is not globally optimal, but it is fast and works well in practice.

We rely on the recursion to repair locally suboptimal choices — if the root split was imperfect, deeper splits compensate.

3.4 Tiny Worked Example — Predicting House Price

Training data ($n = 6$ houses, one predictor sqft):

sqft	price (\$k)
900	180
1100	195
1300	220
1800	320
2200	360
2500	390

At the root, try split $\text{sqft} \leq 1500$:

- Left ($\text{sqft} \leq 1500$): prices $\{180, 195, 220\}$, $\bar{y}_L = 198$.

3.4 Tiny Worked Example — Predicting House Price 2

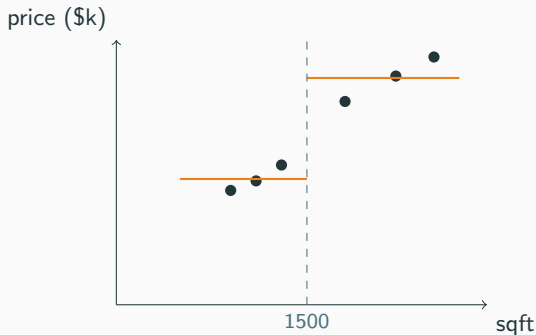
- Right ($\text{sqft} > 1500$): prices $\{320, 360, 390\}$, $\bar{y}_R = 357$.

Each region is much more homogeneous than the parent.

The fitted tree predicts \$198k for small houses and \$357k for big ones.

Two leaves; one decision. Could keep splitting deeper.

3.4 Tiny Worked Example — Predicting House Price 3



Solid points are training data. The orange step-function is the tree's fitted $\hat{f}$. Two leaves $\Rightarrow$ two horizontal segments.

3.5 When to Stop?

Without stopping rules, the tree keeps splitting until every leaf contains a single observation. Training error: zero. Test error: awful (overfit).

Two simple stopping rules:

- **Max depth**: don't grow deeper than D levels.
- **Min samples per leaf**: don't split a node with fewer than m observations.

Each rule has a tuning parameter (D , m). Pick by **cross-validation** (L13).
“Computer does it.”

4. Strengths and Weaknesses of a Single Tree

4.1 What Trees Do Well

- **Interpretability.** A tree is a flowchart. Even non-technical readers understand it.
- **Non-linearity for free.** No need to specify polynomial bases or interactions — the tree discovers them.
- **Mixed predictors.** Numeric, categorical, binary — all handled with the same machinery.
- **No standardisation.** Splits are based on thresholds, so units and scales don't matter (unlike LASSO).
- **Robust to outliers.** An outlier ends up in one leaf; doesn't pollute the fit elsewhere.
- **Universal.** Works for both regression and classification with one algorithm.

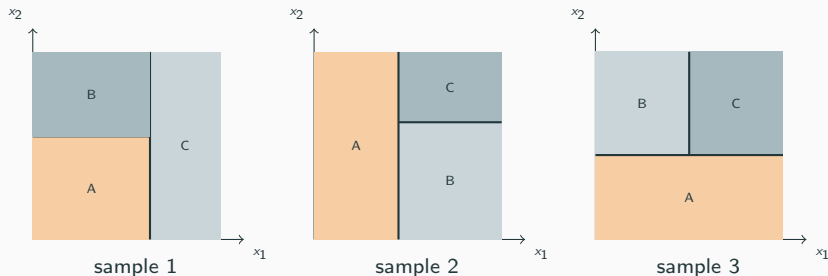
4.2 The Big Weakness — High Variance

Trees are notoriously **unstable**: a small change in the training data can produce a very different tree.

Why? The greedy split at the root depends sensitively on the data. Once the root split changes, the entire downstream tree changes too.

Three bootstrap resamples of the same dataset partition the (x_1, x_2) plane in three visibly different ways:

4.2 The Big Weakness — High Variance 2



Same underlying data, three different trees, three different prediction maps. A fresh point dropped on each tree can land in region A, B, or C depending on the tree — predictions disagree.

4.2 The Big Weakness — High Variance 3

In the language of L10 §3 / L13: a single tree has **low bias** (can fit anything) and **high variance** (different data $\Rightarrow$ different tree).

4.3 What If We Average Many Trees?

Here's the leap. Recall from L3 §3.4: averaging independent noisy estimates reduces variance.

Idea: build **many** trees, each on a slightly different version of the data, and **average** their predictions.

- Regression: average the numeric predictions across trees.
- Classification: majority vote across trees.

Each individual tree is high-variance. The *average* of B trees is much less variable — and turns out to predict dramatically better.

This is the bridge from a single tree to a forest.

5. From Trees to a Forest — Bagging

5.1 Recall — The Bootstrap (L13)

From L13 §8: a **bootstrap sample** is drawn from the original n observations *with replacement*.

In L13 we used the bootstrap to estimate standard errors. Now we use it for a completely different purpose: to generate the “slightly different versions of the data” we need to fit many different trees.

The forward pointer L13 §13.3 promised is now delivered.

5.2 Bagging — Bootstrap Aggregation

Algorithm: bagging

Fix B (the number of trees, e.g. $B = 500$). For $b = 1, \dots, B$:

1. Draw a bootstrap sample of size n from the training data.
2. Grow a tree $\hat{T}_b$ on this resample (deep, no pruning).

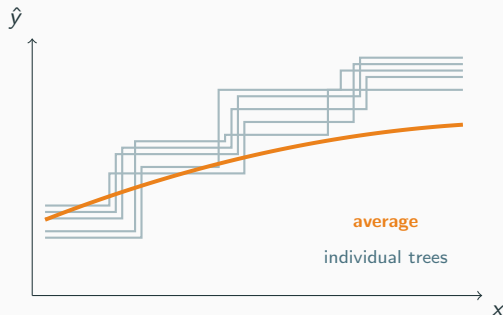
Prediction at a new $\mathbf{x}_{\text{new}}$:

- Regression: $\hat{y}_{\text{bag}}(\mathbf{x}_{\text{new}}) = \frac{1}{B} \sum_{b=1}^B \hat{T}_b(\mathbf{x}_{\text{new}})$.
- Classification: $\hat{y}_{\text{bag}}(\mathbf{x}_{\text{new}}) = \text{majority vote among } \{\hat{T}_b(\mathbf{x}_{\text{new}})\}$.

“Bagging” = Bootstrap aggregating. Same algorithm whether the target is a number or a class label — only the aggregation step differs.

5.3 Why Bagging Works — Variance Reduction

Plain-English version. One tree gives a wobbly prediction. 500 wobbly trees averaged together give a steady prediction. Same idea as “the wisdom of the crowd”: many noisy guesses average out to something accurate.



5.3 Why Bagging Works — Variance Reduction 2

Each thin grey step-function is one tree. The thick orange curve is their average. Even though no single tree is smooth or accurate, the average is.

5.3 Why Bagging Works — Variance Reduction 3

Why does this work? Suppose each tree gives a prediction with variance σ^2 (the spread of those grey step-functions). If the trees were **independent**, the average across B trees has variance

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{T}_b(\mathbf{x}_0)\right) = \frac{\sigma^2}{B}.$$

This is the exact same formula as $\text{Var}(\bar{Y}) = \sigma^2/n$ from L3 §3.4. Average 100 independent noisy quantities and the spread shrinks by a factor of 100.

Bias is unchanged — the average has the same bias as a single tree. So we keep the flexibility (low bias) and dramatically reduce the wobble (variance).

5.3 Why Bagging Works — Variance Reduction 4

Catch. Bagged trees are *not* truly independent. They are all fit on overlapping bootstrap samples and tend to pick the same dominant split at the root. When the trees are correlated, the formula σ^2/B is too optimistic.

The variance reduction is still real — just less than ideal. How can we make the trees more independent of each other? That's the random-forest twist (next).

6. Random Forests

6.1 The Twist — Random Feature Subsets

The problem with plain bagging. Imagine the data has one super-strong predictor (say `credit_score` for loan default). Every bagged tree, no matter which bootstrap sample, will pick that predictor for its root split. All 500 trees look basically the same. Averaging similar trees doesn't reduce variance much — there's nothing to cancel out.

Random forest fix. At each split, only let the tree “see” a random subset of m predictors out of the full p . The other predictors are hidden for that one split.

Now `credit_score` can only be considered when it happens to be among the random m predictors at the current node. Other trees, at their root, will be forced to split on some other predictor instead.

6.1 The Twist — Random Feature Subsets 2

Result: the trees in the forest are **different** from each other — they use different predictors at different places. Different trees make different mistakes, and averaging different mistakes really does cancel out.

6.1 The Twist — Random Feature Subsets 3

Typical defaults (set by software, no need to tune):

- Regression: $m \approx p/3$.
- Classification: $m \approx \sqrt{p}$.

Example: $p = 25$ predictors. Classification forest uses $m = \sqrt{25} = 5$ randomly chosen predictors per split. At each node, the other 20 predictors are temporarily invisible.

Intuition check. Why not pick $m = 1$ (extreme diversity)? Trees would become too random; each tree would be terrible. Why not pick $m = p$ (no random subset)? Then we're back to plain bagging. The defaults $\sqrt{p}$ and $p/3$ sit in the sweet spot.

6.2 The Random-Forest Algorithm

Algorithm: random forest

Fix B (number of trees) and m (number of features per split). For $b = 1, \dots, B$:

1. Draw a bootstrap sample of size n from the training data.
2. Grow a tree $\hat{T}_b$, but at every split consider only a fresh random subset of m of the p predictors.
3. Don't prune.

Aggregate predictions across $\hat{T}_1, \dots, \hat{T}_B$ exactly as in bagging.

Two sources of randomness:

- **Row randomness**: each tree gets a bootstrap sample of the rows.
- **Column randomness**: each split gets a random subset of columns.

6.2 The Random-Forest Algorithm 2

Together they decorrelate the trees and give the random forest its remarkable predictive accuracy.

6.3 Hyperparameters

Three main tuning parameters:

- B (number of trees). More is generally better; diminishing returns past a few hundred. Not really a tuning parameter — bigger is safer.
- m (features per split). Defaults $\sqrt{p}$ (classification) and $p/3$ (regression) usually work well. Can be tuned by CV.
- Tree depth / min samples per leaf. Often left generous; random-forest variance reduction protects against overfitting.

All tunable parameters are picked by **cross-validation** (L13 §4 recipe).

“Computer does it.”

6.4 Strengths and Weaknesses

Strengths

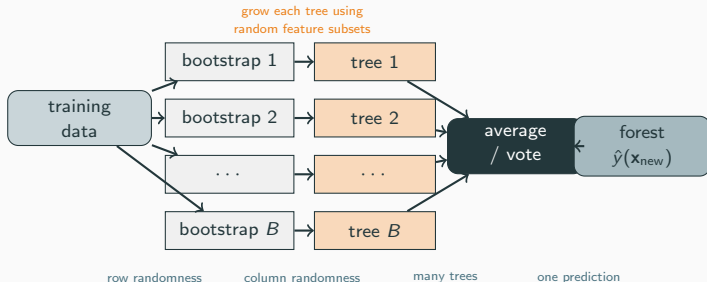
- Excellent out-of-the-box accuracy on tabular data.
- Inherits everything trees do well (non-linear, mixed predictors, no standardisation).
- Handles many predictors well; “free” implicit variable importance.
- Universal across regression and classification.

6.4 Strengths and Weaknesses 2

Weaknesses

- **Black box.** $B = 500$ trees averaged together is hard to interpret. We lose the single-tree flowchart.
- **Computational cost.** B trees instead of one.
- Less suited to extrapolation outside the training predictor ranges (piecewise-constant predictions don't extrapolate).

6.5 The Whole Pipeline in One Picture



The whole random forest in one line. Take your data, make B bootstrap copies of it, grow a tree on each (with random feature subsets at each split), and aggregate the trees' predictions. Done.

7. Summary

7.1 The Course in One Slide

We end where the field of statistical learning ends, with a flexible, high-accuracy, general-purpose method. The whole course was the path to this point:

- **L1–L5.** Probability, statistics, sampling distributions — the foundation.
- **L6–L9.** OLS for regression: one global linear formula, fit by minimising RSS.
- **L10–L11.** LASSO/ridge and polynomial regression: same OLS template, with regularisation or non-linear basis.
- **L12.** Logistic regression: same loss-minimisation template, retargeted to a binary response.
- **L13.** Cross-validation (pick tuning parameters) and the bootstrap (quantify uncertainty without formulas).

7.1 The Course in One Slide 2

- L14. Decision trees and random forests: drop the global formula entirely; partition space adaptively, average many bootstrap-trained trees.

7.2 Take-Aways

Three ideas worth remembering:

- A **decision tree** is a sequence of yes/no questions that partitions predictor space into rectangular regions, fitting a constant in each.
- A **single tree** has low bias but high variance; small data perturbations produce very different trees.
- A **random forest** fits B decorrelated trees on bootstrap samples (with random feature subsets at each split) and averages. Bias unchanged; variance dramatically reduced.

Random forests — with cross-validation for tuning — are one of the most widely-used methods in modern data science.